E7: Introduction to Computer Programming
Final Exam – Version A

Thursday, December 19

Full Name

Student ID Number

Instructions:

- This includes both Part I (Multiple Choice) AND Part II (Fill in the Blanks).
- Write your name above AND on the bubble sheet.
- Mark the correct version letter on the bubble sheet.
- This exam is DOUBLE-SIDED. Make sure to read both sides of each page.
- You have 3 hours to complete this exam.
- You may use two double-sided sheets of notes.
- The exam is worth a total of 300 points.
- For questions with circular bubbles, you should select exactly *one* choice.
  - ◯ You may only chose one option!
  - ◯ Or this one, but not both!
  - ◯ Or this one, but no more than one!
- For questions with square boxes, you may select *multiple* choices.
  - ☐ You can select this one!
  - ☐ You can select this one too!
  - ☐ You can select one, two, or all three of these!
- Only include your FINAL answers in the designated boxes. Any additional text will count as part of your answer and may result in a reduced grade.
- At the end of the exam, hand in the completed bubble sheet AND the exam.

Good luck!

Question 1 (150 points)

There are 50 multiple choice questions, each worth 3 points, for a possible total of 150 points.

- Mark your answers on the bubble sheet with pen or pencil.
- For Question 1, ONLY answers on the bubble sheet will be graded.
- Multiple bubbles, incomplete bubbles, or stray marks will be marked incorrect.
- There is **ONE and only one** correct choice for each question.
- You may use the space below for scratch work.

1. What will be displayed after running the following Python code?

```
x = 10
if x > 5:
    print("YOU")
elif x != 0:
    print("ROCK")
```

A. `YOU`
`ROCK`

B. `YOU`

C. `Error`

D. `Nothing`

E. `ROCK`

2. What will the following Python function return when called as `mystery(4)`?

```
def mystery(limit):
    for i in range(1, limit):
        if i == 2:
            continue
        if i % 2 == 0:
            return i
    return 0
```

A. `2`

B. `4`

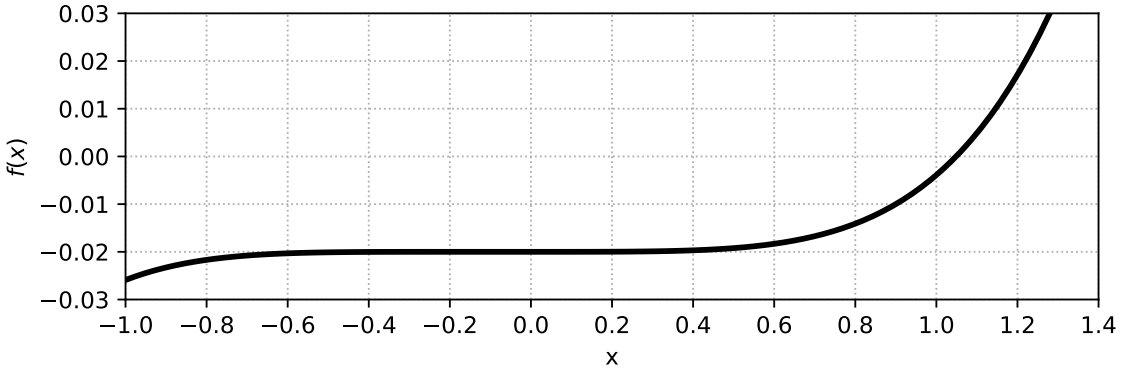
C. `3`

D. `0`

E. `1`

3. Assume that a root estimate is acceptable if the function value at that estimate is within a tolerance of 0.01. Which of the following is an acceptable root estimate for the function below?

A. 0.0
B. 1.0
C. 1.2
D. -1.0
E. 0.8



4. In numerical differentiation, the central difference method calculates the derivative by

   A. using only the previous point
   B. using only the next point
   C. using both the previous and next points
   D. using the central point
   E. averaging all points

5. Why are numerical methods typically implemented in computer programming?

   A. Programming allows us to obtain exact solutions to mathematical problems
   B. All of the options
   C. Programming converts theoretical mathematical equations into graphical representations
   D. Programming allows for instantaneous calculation of complex mathematical problems
   E. They require repetitive computations that can be automated through programming

6. What is a limitation of using smaller step sizes to increase the accuracy of numerical methods?

   A. Smaller step sizes improve accuracy without any computational constraints
   B. Smaller step sizes reduce computation time
   C. Smaller step sizes require the exact mathematical function, which may not be practical
   D. Smaller step sizes could lead to larger errors
   E. Smaller step sizes are only useful for complex numerical methods

7. What will be the final value of `vector` after executing the following Python code?

```
vector = [1, 3, 5, 2, 5, 0]
for i, v in enumerate(vector):
    vector[v] = i
```

   A. `[5, 0, 3, 2, 5, 4]`
   B. `[1, 3, 5, 2, 5, 0]`
   C. None of the options
   D. `[1, 0, 3, 2, 5, 4]`
   E. `[1, 0, 3, 1, 5, 4]`

8. Which of the following is NOT of type `str` in Python.

   A. `'1' + '1'` B. `"  "` C. `"engin"` D. `I <3` E. `str(7)`

9. Let `f` be a function object you want to integrate and `a, b, n` be three scalar values representing the lower integration bound, upper integration bound, and number of subintervals, respectively. What integration method corresponds to this function?

```
import numpy as np
def numerical_integration(f, a, b, n):
    h = (b - a) / n
    x = np.arange(b, a, -h)
    return sum([f(i) for i in x]) * h
```

A. Right Riemann B. Left Riemann C. Trapezoid rule D. Simpson's Rule E. Midpoint rule

10. Which expression computes the element-wise product of two NumPy arrays `a` and `b`?

A. `a @ b` B. `a ** b` C. `a .* b` D. `a * b` E. `a x b`

11. What type of error does the following Python code demonstrate?

```
for i in range(5):
print(i)
```

A. Truncation Error
B. Logical Error
C. None of the options
D. Syntax Error
E. Runtime Error

12. Why does `0.2 + 0.2 == 0.4` return `True` whereas `0.2 + 0.2 + 0.2 == 0.6` return `False`?

A. This is not possible; either both should return `True` or both should return `False`
B. Because Python uses integer arithmetic for these calculations
C. Because of truncation errors
D. Because `==` checks if the expressions are approximately equal to one another
E. Because of the accumulation of round-off error

13. Which of the following statements about the Newton-Raphson method is False?

A. It generally converges faster than the Bisection method
B. It may fail if the derivative of the function equals zero
C. It requires the derivative of the function or an estimate of it
D. It is more accurate than the Bisection method
E. It can be sensitive to the initial guess

14. What does the following MATLAB code display? Assume that `arr` is a defined vector. Note that `max()` is a MATLAB function that returns the largest value(s) in an array.

```
y = 1:numel(arr);
disp(y(arr == max(arr)))
```

A. A logical array

B. The maximum value(s) in the vector `arr`

C. `Error`

D. None of the options

E. The indices of the maximum value(s) in the vector `arr`

15. Which Python expression will NOT create the following sequence: `5, 2, 5, 2, 5, 2`. Assume that you have already imported NumPy as `np`.

A. `np.array([2, 2, 2, 2, 2, 2]) + np.array([3, 0, 3, 0, 3, 0])`

B. `[5, 2] * 3`

C. `[2, 2, 2, 2, 2, 2] + [3, 0, 3, 0, 3, 0]`

D. All the options create the desired sequence

E. `np.array([5, 2, 5, 2, 5, 2])`

16. Which example was NOT mentioned in lecture when discussing differential equations?

A. The age of organic material using carbon dating

B. The motion of a satellite orbiting Earth

C. The amount of salt in two connected tanks

D. The speed of a tsunami wave

E. The motion of a mass-spring-damper system

17. What will be displayed after running the following Python code?

```
total = 1
for i in range(2):
    Total *= 2
print(Total)
```

A. `4` B. `Error` C. `2` D. `1` E. `0`

18. Which slicing operation extracts the second column from a 2D NumPy array named `arr`?

A. `arr[2, :]` B. `arr[1, :]` C. `arr[:, 1]` D. `arr[:, 2]` E. `arr[1]`

19. What will be the final value of `count` after executing the following Python code?

```
count = 0
x = 1
while x < 10:
    if x % 2 == 0:
        x += 3
    else:
        x *= 2
    count += 1
```

A. `4` B. `3` C. `0` D. `1` E. `2`

20. A system of linear equations $A\mathbf{x} = \mathbf{b}$ has a rectangular coefficients matrix A. Which of the following MATLAB expressions must be <u>True</u> for the system to have a unique solution?

A. None of the options

B. `rank([A,b]) == rank(A) & rank(A) == length(b)`

C. `rank([A,b]) == rank(A) & rank(A) == numel(A) / length(b)`

D. `rank([A,b]) == rank(A) & rank(A) > length(b)`

E. `rank([A,b]) == rank(A) & rank(A) < length(b)`

21. Which of the following statements about defining functions in MATLAB is <u>False</u>?

A. The output parameters are specified in the function definition

B. A colon `:` should not be included at the end of the function definition

C. Multiple outputs of a function can be assigned to one variable when the function is called

D. A function is defined using the `function` keyword

E. There is no direct way to assign default arguments in the function definition

22. How can you swap the values of two variables `a` and `b` in Python?

A.
```
a, b = b, a
```

B.
```
a = b
b = a
```

C.
```
a = b
b = a - b
```

D.
```
swap(a, b)
```

E.
```
a = a + b
b = a - b
a = b - a
```

23. Which of the following is NOT a limitation of using the MATLAB expression `inv(A) * b` to obtain a solution to a system of linear equations $A\mathbf{x} = \mathbf{b}$?

    A. It is computationally inefficient for large systems of equations
    B. None of the options
    C. It requires that A be a square matrix
    D. It gives an approximate solution when a unique solution does not exist
    E. It can be numerically unstable

24. What is the purpose of using Taylor series in numerical methods?

    A. To solve systems of linear equations
    B. To eliminate errors in numerical computation
    C. None of the options
    D. To approximate functions using simpler polynomial expressions
    E. To compute exact solutions to differential equations

25. What will be displayed after running the following Python code?

```
a = 1
b = 2
print(a < b + 2 * (a < b))
```

    A. `2` B. `True` C. `Error` D. `3` E. `False`

26. What will be the final value of `x` after executing the following Python code?

```
x = 10
def update():
    return x + 5
x = 20
update()
```

    A. `5` B. `Error` C. `20` D. `10` E. `25`

27. Which of the following methods would give exactly the same solution for the differential equation $y' + 3y - 2 = 0$ when using the same step size h?

    A. Euler's and Midpoint methods
    B. Midpoint and Heun's methods
    C. Euler's and Heun's methods
    D. Euler's and Fourth-Order Runge-Kutta methods
    E. Heun's and Fourth-Order Runge-Kutta methods

28. Which of the following correctly uses an `if` statement in MATLAB?

A.
```
if x > 10:
    x = x + 2
```

B.
```
if x > 10
x = x + 2
```

C.
```
if x > 10:
    x = x + 2
end
```

D.
```
if x > 10
x = x + 2
end
```

E.
```
if x > 10:
x = x + 2
end
```

29. What will be displayed after running the following Python code?

```
x = 4
y = 3
if x > 5:
    if y > 2:
        print("I")
    else:
        print("LOVE")
else:
    if y > 2 or x > 5:
        print("ENGIN")
    else:
        print("7")
```

A. `7` B. `I` C. `ENGIN` D. `Error` E. `LOVE`

30. What will be the final value of `data` after executing the following Python code?

```
data = [12, 10, 24, 20]
for i in range(len(data)):
    data[i] = i
```

A. `[1, 2, 3, 4]`

B. `[12, 10, 24, 20]`

C. `Error`

D. None of the options

E. `[0, 1, 2, 3]`

31. To solve the differential equation $y''' + y' + 3y^4 - 2 = 0$ using the methods we discussed in class, which of the following approaches is correct?

    A. Convert it to a system of three, first-order differential equations

    B. Convert it to a system of two, first-order differential equations

    C. It is not possible to solve this differential equation using any of the methods discussed

    D. Plot the function and see where it intersects the x-axis

    E. Convert it to a system of four, first-order differential equations

32. Which of the following is NOT a practical measure of the error when estimating the root of a function $f(x)$ in real-world applications? Let x_r represent the exact root and x_i represent the estimate at iteration i.

    A. $|x_r - x_i|$

    B. $|x_{i+1} - x_i|$

    C. $|f(x_i)|$

    D. $|x_r - x_i|$ and $|x_i|$

    E. $|x_i|$

33. Which of the following statements about tuples in Python is True?

    A. Tuples are mutable

    B. The first element in a tuple has an index of 1

    C. Tuples are only for strings

    D. Tuples are defined using square brackets

    E. Tuples can contain elements of different data types

34. What will be the final value of `result` be after executing the following Python code?

```
p = 2
q = 3
result = (p > q) and (p + 2 < q) or (p == q or p * 2 > q)
```

    A. `2` B. `False` C. `True` D. `Error` E. `5`

35. What will the following Python function return when called as `func(3, c=4)`?

```
def func(a, b=5, c=10):
    return a + b + c
```

    A. `18` B. `12` C. `17` D. `15` E. `Error`

36. At what time did our lecture occur? All listed times are in the afternoon (PM).

A. 3:00 – 4:00 B. 5:00 – 6:00 C. 2:00 – 3:00 D. 1:00 – 2:00 E. 4:00 – 5:00

37. Which of the following statements about Python functions is False?

A. A function does not need to have a `return` statement
B. A function can return multiple values
C. Functions in Python can have default arguments
D. Variables declared inside a function are global by default
E. A function can call itself

38. Which of the following statements about number representation in computers is False?

A. There is a non-zero gap between consecutive representable numbers
B. There is a maximum representable number
C. None of the statements are False
D. There is a finite number of representable numbers
E. There is a minimum representable number

39. Which of the following statements is False when approximating a function $f(x)$ using a Taylor series around $a = 5$, truncated after $n = 3$?

A. The error is proportional to x^3
B. The estimated error at $x = 0$ is equal to that at $x = 10$
C. The approximation is exact at $x = 5$
D. The approximation becomes less accurate as x moves farther from 5
E. None of the options

40. What will be the final value of `result` after executing the following MATLAB code?

```
result = 1;
for i = 1:2:7
    result = result + i;
end
```

A. `16` B. `Error` C. `10` D. `17` E. `1`

41. Which of the following is NOT a valid Python variable name?

A. `my_var` B. `myVar` C. `MyVar` D. `_hidden` E. `2ndVar`

42. Which of the following best describes the general idea used in numerical differentiation?

    A. Approximating the derivative of a function using its values at discrete points
    B. Evaluating the derivative analytically through symbolic computation
    C. Approximating the derivative of a function using different shapes, such as rectangles
    D. Solving differential equations by finding numerical solutions
    E. Using regression to approximately compute derivatives

43. Which package includes functions that can be used for obtaining numerical solutions for tasks such as finding roots, integration, differential equations, among others?

    A. `math` B. `numpy` C. `csv` D. `matplotlib` E. `scipy`

44. Which type(s) of ordinary differential equations can Euler's, Midpoint, Heun's, and Fourth-Order Runge-Kutta methods be used to solve?

    A. Fourth-order ordinary differential equations
    B. First-order ordinary differential equations
    C. Nonlinear ordinary differential equations
    D. Systems of first-order ordinary differential equations
    E. All of the listed options

45. What will the following Python function return when called as `fun(9)`?

```
def fun(n):
    if n > 10:
        return n - 1
    else:
        return n + fun(fun(n + 1))
```

    A. `Error` B. `19` C. `20` D. `10` E. `9`

46. Which of the following best explains truncation errors in Taylor series approximations?

    A. Computers cannot store exact values of numbers
    B. The approximations are only valid for small values of x
    C. Higher-order terms in the series are ignored
    D. None of the options
    E. The function being approximated is discontinuous

47. What will be displayed after running the following Python code?

```
result = 0
def triple(n):
    result = n * 3
result = triple(2) + 1
print(result)
```

A. `7`

B. `3`

C. `0`

D. None of the options

E. `6`

48. Which of the following statements about `try-except` blocks is True?

A. The `try` block should contain code that you expect may cause an exception

B. They can only handle one type of exception at a time

C. Specifying the error type after `except` is required

D. Using `try-except` will always prevent a code from crashing

E. They can handle any type of error

49. Which technique would you use to estimate the area under a function with high curvature?

A. Midpoint rule

B. Simpson's rule

C. Trapezoidal rule

D. Left Riemann sum

E. Fourth-Order Runge-Kutta method

50. Which of the following is necessary to ensure that a recursive function terminates?

A. An `end` keyword

B. A `break` statement

C. A base case

D. A small tolerance value

E. A counter variable

Question 2 (6 points)

This is the beginning of the fill in the blank questions.
Question 2 is only worth 6/300 points (2%), and some might find it challenging.
Finish the rest of the exam first!

An arithmetic sequence is a series of numbers in which each term is obtained by adding a constant value, known as the common difference d, to the previous term. In general, an arithmetic sequence that starts at a with a common difference d is represented as: a, $a+d$, $a+2d$, $a+3d$, ..., $a+(n-1)d$, where n is the number of terms in the sequence. For example, consider the sequence $1, 4, 7, 10$. This is an arithmetic sequence that starts at $a = 1$ with a common difference of $d = 3$ and $n = 4$ terms.

Consider the Python function `sum_rec` that takes three scalar arguments: `a` (float representing the first term), `d` (float representing the common difference), and `n` (positive integer representing the number of terms). The function should return the sum of the `n` terms of the arithmetic sequence using recursion. Any solutions that do not use recursion to compute the sum will receive 0 points.

```
def sum_rec(a, d, n):
    """Return the sum of the n terms in an arithmetic sequence
    >>> sum_rec(1, 3, 4) # 1 + 4 + 7 + 10
    22
    >>> sum_rec(1, 4, 3) # 1 + 5 + 9
    15
    """
    if n == 1:
        ______(a)______
    else:
        ______(b)______
```

You will implement the function by filling in the blanks. You may only write one statement on each line. You may not use ; to place two statements on the same line. You may not use any functions, other than the recursive function. You may not modify the level of indentation of any line.

(a) (1 pt) Fill in blank (a).

(b) (5 pt) Fill in blank (b).

Question 3 (25 points)

The California Department of Fish and Wildlife is studying salmon populations to ensure their long-term sustainability. Let P represent the population of salmon at any given time. The rate of change in the salmon population can be modeled using the logistic growth equation:

$$\frac{dP}{dt} = rP\left(1 - \frac{P}{K}\right)$$

where r is the growth rate and K is the carrying capacity, both of which are scalar values.

(a) (3 pt) Which of the following about the above equation is(are) True? Select ALL that apply.

- ☐ It is a nonlinear differential equation
- ☐ It is an ordinary differential equation
- ☐ It is a partial differential equation
- ☐ It is a first-order differential equation
- ☐ It is a linear differential equation
- ☐ It is a second-order differential equation

Define a Python function named `dP` which takes three scalar values `P, r, K` representing the current population, growth rate, and carrying capacity, respectively. Assign `K` a default value of 10000. The function should return a scalar representing the value of dP/dt (the right-hand side of the above equation). You will define the function by filling in the blanks.

```
def ______(b)______:
    return ______(c)______
```

(b) (5 pt) Fill in blank (b).

```
def
```

(c) (4 pt) Fill in blank (c).

```
return
```

Next, you will implement Heun's method to solve the given differential equation. This method approximates the solution by calculating two slopes: k_1, the derivative at the current point, and k_2, the derivative at the predicted point based on k_1. These slopes are then averaged to obtain an estimate of the solution. Write a Python function named `Heun`, which takes six scalar arguments `t0, P0, r, K, h, n` representing the initial time, initial population, growth rate, carrying capacity, step size, and number of steps, respectively. The function should implement Heun's method to solve the logistic equation for the salmon population. It should return the final time $t_f = t_0 + nh$ and the estimated population P at the final time.

```
def Heun(t0, P0, r, K, h, n):
    ti = t0       # current time step
    Pi = P0       # solution at current time step
    while ti < t0 + n * h:
        k1 = ______(d)______    # derivative at the current point
        k2 = ______(e)______    # derivative at the predicted point
        Pi = ______(f)______
        ti += h
    return ti, Pi
```

You will implement the function `Heun` by filling in the blanks. You may only write one statement on each line. You may not use ; to place two statements on the same line. The function `dP`, which returns dP/dt based on the values of `P, r, K` is already defined in the global scope. You may not use any other function or methods. You may not modify the level of indentation of any line.

(d) (3 pt) Which of the following options could fill in blank (d)?

- ◯ `dP(Pi[i])`
- ◯ `dP(P0, K, r)`
- ◯ `dP(P0, r, K)`
- ◯ `dP(Pi, K, r)`
- ◯ `dP(Pi[i], r, k)`
- ◯ `dP(Pi[i], K, r)`
- ◯ `dP(P0)`
- ◯ `dP(Pi)`
- ◯ `dP(Pi, r, K)`

(e) (5 pt) Fill in blank (e).

```
k2 =
```

(f) (5 pt) Fill in blank (f).

```
Pi =
```

Question 4 (55 points)

A fixed point problem involves finding a value x such that a function $g(x)$ satisfies $g(x) = x$. You are interested in finding the value of x where the cosine function equals the value itself: $\cos(x) = x$.

(a) (3 pt) Which of the following methods is most appropriate for estimating the value of x?

- ◯ numerical integration
- ◯ numerical differentiation
- ◯ numerical ODE solvers
- ◯ numerical root finding
- ◯ Taylor series
- ◯ regression

Let $f(x) = \cos(x) - x$. One challenge is that the cosine function cannot be directly evaluated. The cosine function can be approximated using Taylor series around $a = 0$ as:

$$\cos(x) = 1 - \frac{1}{2!}x^2 + \frac{1}{4!}x^4 - \frac{1}{6!}x^6 + \cdots = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n)!}x^{2n}$$

Complete the following Python function, which takes a scalar `x` and a positive integer `N`, and returns an approximation of $f(x) = \cos(x) - x$ using the Taylor series expansion up to $n = N$. Assume `fact(n)` is already defined, which returns the factorial of an integer `n`.

```
def f(x, N):
    """
    Return an approximate value of f(x) = cos(x) - x up to n = N (inclusive).
    """
    est = 1    # initialize est, an estimate of cos(x), as 1
    for n in ______(b)______:
        ______(c)______
    return ______(d)______
```

(b) (3 pt) Which option could fill in blank (b)?

- ◯ `range(N + 1)`
- ◯ `int(N)`
- ◯ `range(1, N)`
- ◯ `len(N - 1)`
- ◯ `range(N - 1)`
- ◯ `range(1, N - 1)`
- ◯ `len(N + 1)`
- ◯ `len(N)`
- ◯ `range(1, N + 1)`

(c) (6 pt) Fill in blank (c).

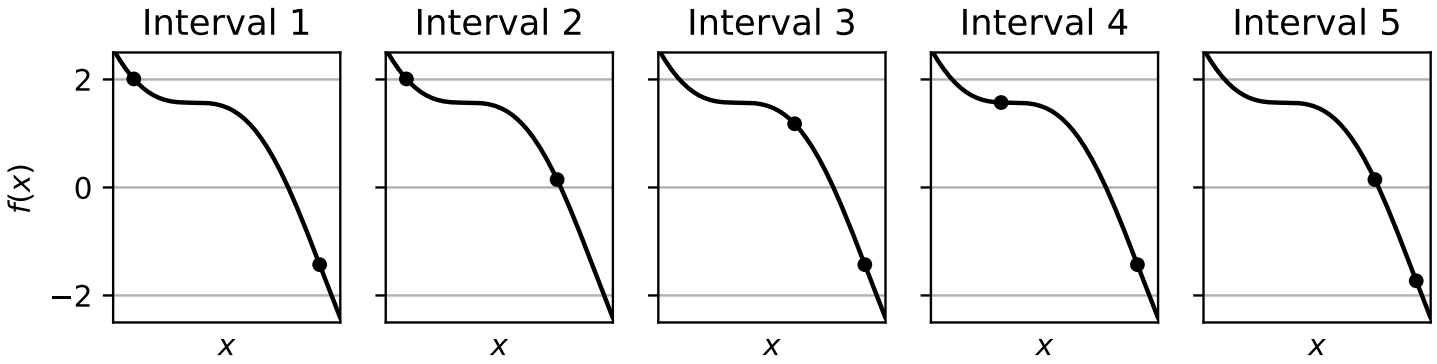
(d) (3 pt) Which option could fill in blank (d)?

- ◯ `est + x`
- ◯ `est`
- ◯ `est - x`
- ◯ `cos(x) - x`
- ◯ `1 + est - x`
- ◯ `1 + est`
- ◯ `f(x, N - 1)`
- ◯ `1 + est + x`

(e) (4 pt) Which of the following represents the absolute value of the estimated error in approximating $\cos(x)$ using the Taylor series expansion up to $n = N$?

- ○ $\frac{x^{N+1}}{(N+1)!}$
- ○ $\frac{x^{2(N+1)}}{(2(N+1))!}$
- ○ $\frac{x^{2N}}{(2N)!}$
- ○ $\frac{x^{2N+1}}{(2N+1)!}$
- ○ $\frac{1}{(2(N+1))!}$
- ○ $\frac{1}{(2(N-1))!}$
- ○ $\frac{x^{2(N-1)}}{(2(N-1))!}$
- ○ $\frac{1}{(2N)!}$

Next, you want to use the Bisection method to find the value of x where the cosine function equals the value itself: $f(x) = \cos(x) - x = 0$. You are considering different initial intervals $[a, b]$, which are indicated by the black filled circles in the figure below.



(f) (3 pt) Which initial interval fails to converge to an acceptable root?

○ Interval 4 ○ Interval 1 ○ Interval 3 ○ Interval 5 ○ Interval 2

(g) (3 pt) Which initial interval is likely to converge the slowest to an acceptable root?

○ Interval 4 ○ Interval 3 ○ Interval 2 ○ Interval 5 ○ Interval 1

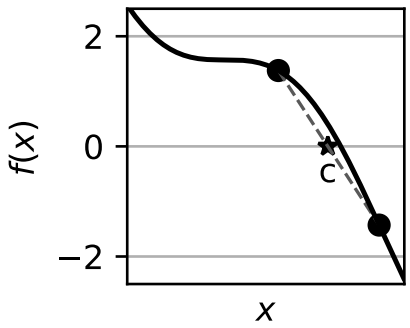
(h) (3 pt) Assume you have a continuous function $f(x)$ that has at least one root. Which of the following statements about the Bisection method is(are) True? Select ALL that apply.

- ☐ Increasing the tolerance will generally increase the accuracy of the estimated root
- ☐ None of the options are True
- ☐ It always converges to a root that is between an acceptable initial interval
- ☐ Increasing the tolerance will generally increase the time needed to find an acceptable root
- ☐ One advantage is that the method always works even when only discrete data are available
- ☐ Choosing a narrow initial interval improves the accuracy of the estimated root

Finally, you will implement the False Position method, a root-finding technique similar to the Bisection method. The False Position updates the guess using the intersection of a line connecting the function values at the interval's endpoints with the x-axis. The steps are:

1. Choose an interval $[a, b]$ where $f(x)$ changes signs
2. Find the point c where the line connecting $(a, f(a))$ and $(b, f(b))$ intersects the x-axis:

$$c = \frac{af(b) - bf(a)}{f(b) - f(a)}$$



3. If c has an acceptable error, you have found an estimate of the root
4. Else if $f(a)$ and $f(c)$ have opposite signs, there is a root between a and c. Otherwise, there is a root between c and b.
5. Repeat steps 2, 3, and 4 until finding an acceptable root

Write a Python function `FalsePos`, which takes as input a function object `f` whose root needs to be found, two floats `a, b` representing the initial interval $[a, b]$, a positive integer `maxiter` specifying the maximum number of iterations allowed, and a positive float `tol` for the tolerance. For simplicity, assume that the function `f` accepts only one argument, the x-value: `f(x)`.

The function should return three scalars: the estimated root, the absolute value of the function at the estimate, and the number of iterations performed. The function should terminate when the absolute value of the function at the estimate is less than or equal to the tolerance or after `maxiter` iterations, whichever occurs first. Assume that the function has opposite signs at the initial interval.

```
import numpy as np
def FalsePos(f, a, b, maxiter, tol):
    for i in ______(i)______:
        c = ______(j)______
        if ______(k)______:
            ______(l)______
        elif ______(m)______:
            b = c
        else:
            ______(n)______
    return ______(o)______
```

You will implement the function by filling in the blanks. You may only write one statement on each line. You may not use ; to combine multiple statements. You may only use the `np.abs` function for absolute values and the input `f` function. No other functions or methods are allowed unless provided in multiple choice options. You may not modify the level of indentation of any line.

(i) (3 pt) Which option could fill in blank (i)?

- ◯ `len(maxiter + 1)`
- ◯ `len(maxiter - 1)`
- ◯ `str(maxiter)`
- ◯ `range(maxiter + 1)`
- ◯ `range(len(maxiter))`
- ◯ `range(maxiter - 1)`
- ◯ `len(maxiter)`
- ◯ `range(maxiter)`
- ◯ `int(maxiter)`

(j) (3 pt) Which option could fill in blank (j)?

- ◯ `af(b) - bf(a) / f(b) - f(a)`
- ◯ `(a*f(b) - b*f(a)) / (f(b) - f(a))`
- ◯ `(af(b) - bf(a)) / (f(b) - f(a))`
- ◯ `a*f(b) - b*f(a) / f(b) - f(a)`

(k) (5 pt) Fill in blank (k).

```
if
```

(l) (4 pt) Fill in blank (l).

```

```

(m) (4 pt) Fill in blank (m).

```
elif
```

(n) (4 pt) Fill in blank (n).

```

```

(o) (4 pt) Fill in blank (o).

```
return
```

Question 5 (30 points)

Waymo, a developer of autonomous vehicle technology operating in San Francisco and other cities, seeks your help analyzing driving data to enhance their vehicle performance. They provided a dataset of a car's velocity over time, which you imported into Python as two NumPy arrays:

- `t`: an array of the timestamps at irregular intervals, e.g.: `0, 1, 1.2, 1.3, ...`
- `v`: an array of the corresponding velocities at each timestamp, e.g.: `0, 5, 20, 22, ...`

First, they are interested in estimating the acceleration by computing the derivative of velocity. Complete the Python function `differentiation` which takes two NumPy arrays `t, v` of the time and corresponding velocities, respectively. The function should compute the acceleration of the car using the central difference method at the points in `t` for which it can be applied. Your function should return two Numpy arrays `a, time`, where `a[i]` is the acceleration at `time[i]`. You should implement this function without using iteration. Note that `t` may be at irregular intervals.

```
def differentiation(t, v):
    a = ______(a)______
    time = ______(b)______
    return a, time

# Assume t and v are defined. The lines below are for part (c).
a, time = differentiation(t, v)
count = ______(c)______
```

(a) (6 pt) Fill in blank (a).

```
a =
```

(b) (3 pt) Fill in blank (b).

```
time =
```

(c) (4 pt) Fill in blank (c) to calculate `count`, a scalar representing the number of times the car transitions from accelerating (acceleration > 0) to constant velocity (acceleration $= 0$). You may not use ; to place two statements on the same line. You may only use the built-in `sum` function, which returns the sum of a sequence. No other functions or methods are allowed.

```
count =
```

Next, they want to estimate the car's distance traveled over time by integrating the velocity. Complete the Python function `integration` which takes two NumPy arrays `t, v` of the time and corresponding velocities, respectively. The function should compute the total distance traveled at each point in `t` using the trapezoid rule and return it as a NumPy array `x`, where `x[i]` is the total distance traveled up to `t[i]`. Assume the distance starts at 0 at `t[0]`. You should implement this function using iteration. Note that `t` may be at irregular intervals. *Hint: At `t[i]`, the total distance is the sum of the distance at `t[i-1]` and the distance traveled between `t[i-1]` and `t[i]`:*

$$x_i = \int_{t_0}^{t_i} v(t)\,dt = x_{i-1} + \int_{t_{i-1}}^{t_i} v(t)\,dt, \text{ where } v(t) \text{ is the velocity over time}$$

```
import numpy as np
def integration(t, v):
    x = np.zeros(len(t)) # initialize an array of zeros with the same length as t
    for i in ______(d)______:
        ______(e)______
    return x
# Assume t and v are defined. The lines below are for part (f).
x = integration(t, v)
target = ______(f)______
```

(d) (3 pt) Which option could fill in blank (d)?

◯ `range(len(x))` ◯ `len(x) - 1` ◯ `len(x)` ◯ `range(1,len(x))`
◯ `t` ◯ `range(x)` ◯ `v` ◯ `range(1,x)`

(e) (6 pt) Fill in blank (e).

(f) (4 pt) Fill in blank (f) to compute `target`, a scalar representing the first time instance when the distance traveled exceeds 100. You may not use ; to place two statements on the same line. You may not use any functions or methods, built-in or otherwise.

```
target =
```

(g) (4 pt) Why are numerical methods useful in this case?

◯ The velocity data is available only at discrete time intervals
◯ Numerical methods can remove errors from the measurements for better results
◯ Numerical methods are faster than analytical methods
◯ Numerical methods are more accurate than analytical methods

Question 6 (34 points)

(a) (3 pt) Arrange the following numerical differential equation solvers based on their accuracy, from most accurate (smallest error) to least accurate (largest error), for the same step size h:

- ◯ Fourth-Order Runge-Kutta Method, Midpoint Method, Euler's Method
- ◯ Heun's Method, Euler's Method, Fourth-Order Runge-Kutta Method
- ◯ Midpoint Method, Euler's Method, Fourth-Order Runge-Kutta Method
- ◯ Euler's Method, Midpoint Method, Fourth-Order Runge-Kutta Method
- ◯ Fourth-Order Runge-Kutta Method, Heun's Method, Midpoint Method

More accurate numerical methods generally require more computations per step, but they achieve higher accuracy for a given step size. Assume you are comparing two numerical methods:

- Method 1: Error proportional to $\mathcal{O}(h^{k_1})$
- Method 2: Error proportional to $\mathcal{O}(h^{k_2})$, where $k_1 > k_2$

For a given step size h_1 used in Method 1, you are interested in estimating the step size h_2 required for Method 2 to achieve an error less than or equal to that of Method 1. For simplicity, assume that the order of the error actually represents the value of the error. Specifically, h_2 will be computed by repeatedly dividing h_1 by a factor of 2. You need to write a Python function that computes the number of step size divisions by 2 until the error of Method 2 becomes less than or equal to that of Method 1. Refer to the examples below. The code is shown on the next page.

Example 1: `k1=4, k2=2, h1=0.5`

Error Method 1: $h_1^{k_1} = 0.0625$

Divide h_1 by 2 zero times: $h_2 = h_1/2^0 = 0.5 \rightarrow$ Error Method 2: $h_2^{k_2} = 0.25 > 0.0625$

Divide h_1 by 2 one time: $h_2 = h_1/2^1 = 0.25 \rightarrow$ Error Method 2: $h_2^{k_2} = 0.0625 \leq 0.0625$

Return `1`

Example 2: `k1=3, k2=1, h1=0.2`

Error Method 1: $h_1^{k_1} = 0.008$

Divide h_1 by 2 zero times: $h_2 = h_1/2^0 = 0.2 \rightarrow$ Error Method 2: $h_2^{k_2} = 0.2 > 0.008$

Divide h_1 by 2 one time: $h_2 = h_1/2^1 = 0.1 \rightarrow$ Error Method 2: $h_2^{k_2} = 0.1 > 0.008$

Divide h_1 by 2 two times: $h_2 = h_1/2^2 = 0.05 \rightarrow$ Error Method 2: $h_2^{k_2} = 0.05 > 0.008$

Divide h_1 by 2 three times: $h_2 = h_1/2^3 = 0.025 \rightarrow$ Error Method 2: $h_2^{k_2} = 0.025 > 0.008$

Divide h_1 by 2 four times: $h_2 = h_1/2^4 = 0.0125 \rightarrow$ Error Method 2: $h_2^{k_2} = 0.0125 > 0.008$

Divide h_1 by 2 five times: $h_2 = h_1/2^5 = 0.00625 \rightarrow$ Error Method 2: $h_2^{k_2} = 0.00625 \leq 0.008$

Return `5`

Write a Python function `countDivisions` that takes three scalar arguments: `k1` (a positive integer representing the order of Method 1), `k2` (a positive integer representing the order of Method 2), and `h1` (a positive float representing the step size for Method 1). The function should return `count`, the number of times `h1` must be divided by 2 for the error of Method 2 to become less than or equal to the error of Method 1 following the approach shown in the examples. `k1` should be greater than `k2`. Otherwise, the function should return the integer `-999`.

```
def countDivisions(k1, k2, h1):
    ______(b)______
        return -999
    ______(c)______
    ______(d)______
        count = ______(e)______
    return count
```

You will implement the function by filling in the blanks. You may only write one statement on each line. You may not use ; to place two statements on the same line. You may not use any functions or methods, built-in or otherwise. You may not modify the level of indentation of any line.

(b) (4 pt) Fill in blank (b).

(c) (4 pt) Fill in blank (c).

(d) (6 pt) Fill in blank (d).

(e) (4 pt) Fill in blank (e).

```
count =
```

It is possible to calculate analytically the number of divisions by 2 for the error of Method 2 to become exactly equal to the error of Method 1 using the following formula:

$$\frac{(k_2 - k_1) \cdot log(h_1)}{k_2 \cdot log(2)}$$

(f) (4 pt) Which expression(s) can fill blank (f) to return the above value? Select ALL that apply.

```
import math
def exactDivisions(k1, k2, h1):
    return ______(f)______
```

- ☐ `(k2 - k1) log(h1) / k2 log(2)`
- ☐ `k2 - k1 * math.log(h1) / (k2 * math.log(2))`
- ☐ `(k2 - k1) math.log(h1) / k2 math.log(2)`
- ☐ `(k2 - k1) * math.log(h1) / k2 / math.log(2)`
- ☐ `(k2 - k1) * math.log(h1) / k2 * math.log(2)`
- ☐ `(k2 - k1) * math.log(h1) / (k2 * math.log(2))`
- ☐ `(k2 - k1) . math.log(h1) / (k2 . math.log(2))`

(g) (4 pt) Because the equation above is derived so that the error of Method 2 matches exactly the error of Method 1, the function `exactDivisions` returns a float. Your task is to compute the ceiling of this value, which is the smallest integer greater than or equal to the number. For example, the ceiling of 4.0 is 4, of 4.15 is 5, and of 4.95 is 5. Consider the following function:

```
def ceilDivisions(k1, k2, h1):
    exact = exactDivisions(k1, k2, h1)
    return ______(g)______
```

Fill in blank (g) to return the ceiling of `exact`. You may only use the built-in `int` function. No other functions are allowed. You may not use conditional statements.

```
return
```

(h) (5 pt) Finally, assume that based on your previous functions, you found the time required for Methods 1 and 2 to achieve similar accuracy, denoted by `t1` and `t2`, respectively. Define a single-line Python function named `method` which takes two scalar arguments `t1, t2` and returns the integer `2` if Method 2 requires less time than Method 1, and the integer `1` otherwise. Your answer must combine the function definition and its body into a single line of code.

```

```